

Desktop GUI & Client Shell – SonarQube Baseline Report (BNCH-2)

Sub-team 6 · Quality Champion · Day 2-3 Baseline · 19-20 May 2026

1. Scope

This report presents the SonarQube-equivalent static analysis baseline for the eight classes owned by the Desktop GUI and Client Shell work package. Metrics include cyclomatic complexity, cognitive complexity, code smells, duplication, technical debt, and maintainability rating. This report complements the CK metrics baseline and together they form deliverable T2.

2. Summary Table

#	File	LOC	CC Total	Max Method CC	Methods >10 CC	Key Code Smells	Dup %	Debt Est.	Rating
1	CanvassDesktop.cs	1,899	~215	31 (<code>checkSubsetBounds`</code>)	4	God Class, Duplicate Validation, Deep Nesting, Scene Path Coupling, Long If/Else Chain	~8%	~3 d	**D**
2	DesktopPaintController.cs	1,558	~185	18 (<code>Update`</code>)	5	God Class, Systematic Axis Duplication, Long Method, Copy-Paste Bug, FindObjectOfType Coupling	~16%	~4 d	**D**

3	PaintMenuController.cs	371	~42	5	(`ToggleMask`)	0	Scene Path Coupling, Dead/Commented Code, FindObjectOfType	<2%	~5 h	**B**
4	HistogramMenuController.cs	222	~20	5	(`Update`)	0	Repeated Method Calls, Minor Structural Duplication	~4%	~1.5 h	**B**
5	HistogramHelper.cs	102	~5	3	(`CreateHistogramImg`)	0	Cross-Cutting Concern, Empty Lifecycle Methods, Long Param List	0%	~2 h	**A**
6	TabsManager.cs	108	~9	5	(`UpdateActiveTab`)	0	Minor Logic Complexity	0%	~1 h	**A**
7	SourceRow.cs	62	~3	1	(all methods)	0	None significant	0%	<0.5 h	**A**
8	VideoUiManager.cs	439	~30	6	(`ValidateFfmpegPath`)	0	SRP Violation, Dead Field, TODO Debt	<1%	~3.5 h	**B**
	TOTAL	**4,761**	**~509**	—	**9**	—	—	**~67 h**	—	

****Rating scale:**** A ≤ 5% debt ratio · B 6-10% · C 11-20% · D 21-50% · E > 50%

3. High-Complexity Methods (CC > 10)

File	Method	Cyclomatic Complexity
-----	-----	-----
CanvassDesktop.cs	`checkSubsetBounds()`	**31** ●
CanvassDesktop.cs	`_browseMappingFile()`	**19** ●
CanvassDesktop.cs	`IsLoadable()`	**15** ●
CanvassDesktop.cs	`Update()`	**10** ▲
DesktopPaintController.cs	`Update()`	**18** ●
DesktopPaintController.cs	`FillPolygon()`	**15** ●
DesktopPaintController.cs	`GetSlice()`	**13** ●
DesktopPaintController.cs	`DrawOutlineAndGrid()`	**12** ●
DesktopPaintController.cs	`UpdateMaskVoxels()`	**10** ▲

All 9 methods exceeding CC > 10 are concentrated in the two god classes. These are the primary refactoring targets.

4. Top-10 Worst Code Smells

Rank 1 – BLOCKER: God Class (CanvassDesktop.cs)

****Scope:**** Entire class (1,899 lines, 60+ methods)

The class spans FITS I/O, HDU parsing, histogram maths, colour maps, subset bounds, source mapping, paint-mode wiring, stats, and threshold

controls – all in one MonoBehaviour. Every feature change risks silent regressions in unrelated concerns. Zero unit-testable boundaries exist. Exceeds WMC, CBO, and RFC thresholds by 3-5×.

Rank 2 – BLOCKER: God Class (DesktopPaintController.cs)

****Scope:**** Entire class (1,558 lines, 57 methods)

Combines 2D texture slicing, GPU camera setup, polygon selection, mask-voxel writes, colour mapping, scroll-zoom/pan, and UI sync. No seam exists between pure C# logic and Unity APIs, making the class untestable and impossible to mock.

Rank 3 – CRITICAL: Copy-Paste Bug (DesktopPaintController.cs)

****Scope:**** `UpdateMaxValue()` – Line 306

`public void UpdateMaxValue(float value) { minVal = value; }` assigns to `minVal` instead of `maxVal`. This is a silent data-corruption bug – the max threshold never updates via this path, causing wrong colour scaling with no runtime error. Introduced by duplicating `UpdateMinValue()` without editing the assignment. This directly demonstrates the danger of untestable god classes.

Rank 4 – CRITICAL: Systematic Axis Duplication (DesktopPaintController.cs)

****Scope:**** `GetPrevMask`, `UpdateSourceColours`, `ClearAllButton`, `FillPolygon`, `UpdateMaskVoxels`, `UpdateLastMaskVoxels`, `DrawOutlineAndGrid`, `PreviousSlice`

The block ``if(axis==0){...} if(axis==1){...} if(axis==2){...}`` constructing a ``Vector3Int`` voxel coordinate is copy-pasted verbatim across 8 methods (~240 lines). Adding a fourth axis or fixing the coordinate formula requires 8 identical edits. A single ``GetVoxelCoord(int axis, int x, int y, int sliceIndex)`` helper eliminates all eight instances.

Rank 5 – CRITICAL: Duplicate Validation + Long Method (CanvassDesktop.cs)

****Scope:**** ``checkSubsetBounds()`` – CC 31

The same "TryParse → clamp lo → clamp hi → fallback" block is copy-pasted 6 times for XMin/XMax/YMin/YMax/ZMin/ZMax (~160 lines, cognitive complexity ~50). A single ``ValidateBound(TMP_InputField field, int min, int max, int fallback)`` would reduce this to 6 one-liners.

Rank 6 – MAJOR: Long Method (DesktopPaintController.cs)

****Scope:**** ``Update()`` – CC 18, ~110 lines

Per-frame method handles nine keyboard shortcuts, scroll-wheel zoom with UV-rect maths, and colour-map sync. The zoom section alone is 20 lines with 4 branches. Any input behaviour change requires reading the whole method.

Rank 7 – MAJOR: Long If/Else Chain (CanvassDesktop.cs)

****Scope:**** ``_browseMappingFile()`` – CC 19

15-way if/else if chain mapping `SourceMappingOptions` enum values to dropdown assignments (~100 lines). Adding one new mapping type requires editing this chain in two separate loops plus the `Mapping` constructor. Should be a `Dictionary<SourceMappingOptions, Action<SourceRow, TMP\_Dropdown>>`.

Rank 8 – MAJOR: Scene Hierarchy Coupling (CanvassDesktop.cs, PaintMenuController.cs)

****Scope:**** `Start()`, `ShowOutline()`, `ToggleMask()`, `postLoadFileFileSystem()`, `populateColorMapDropdown()`, + ~25 others

30+ locations use 4-6 level chains such as

```
`renderingPanelContent.gameObject.transform.Find("Rendering_container")  
  .gameObject.transform.Find("Viewport")...GetComponent<TMP_Dropdown>()`.
```

These strings are never compile-time checked: renaming any scene GameObject produces a silent NullReferenceException at runtime. All should be `[SerializeField]` references cached in `Start()`.

Rank 9 – MINOR: Repeated Method Calls (HistogramMenuController.cs)

****Scope:**** `UpdateButtonHandler()`, `ResetButtonHandler()`

`getFirstActiveDataSet()` is called 5× in each handler with no local variable caching. Each call re-iterates the entire `dataSets` array. If the active dataset changes between calls mid-frame, the two handlers would operate on different objects.

Rank 10 – MINOR: Cross-Cutting Concern (HistogramHelper.cs)

```
**Scope:** `CreateHistogramImg()`
```

`HistogramHelper` directly calls `histogramMenu.UpdateUI()` and `canvassDesktop.UpdateUI()` on the last two lines, making it a hidden bridge between the VR menu layer and the desktop GUI layer. Neither `HistogramMenuController` nor `CanvassDesktop` can be used without the other being present in the scene. An `OnHistogramUpdated` C# event would decouple all three.

```
---
```

5. Refactoring Priorities

The data points directly to two high-value refactoring targets for the worked examples:

1. ****`checkSubsetBounds` → `ValidateBound()` extraction**** – a clean, self-contained worked example with quantifiable CK delta (WMC drops from ~215 to ~185, method CC from 31 to 1).
2. ****Axis-dispatch pattern → `VoxelCoordinateMapper` value object**** – strongest argument for a typed, testable abstraction in the ViewModel layer, eliminating ~240 lines of copy-paste across 8 methods.

The copy-paste bug in `UpdateMaxValue` should be highlighted in the pitch as a real-world consequence of the god class pattern – the method was clearly duplicated from `UpdateMinValue()` with the body left unedited, and no test exists to catch it.

```
---
```

**Report prepared by Sub-team 6 Quality Champion · Sprint 1 · iDaVIE
Refactoring Assessment 2026**